

1. Introducción

Esta práctica se basa en el código que ya teníamos del proyecto anterior (el mundo con arena, los murciélagos y el gladiador). La idea era tomar esa misma estructura de clases y adaptarla para hacer un juego completamente diferente.

Lo que se propuso fue algo parecido a Animal Crossing, aunque no una copia del juego. Nada de deudas ni tiendas ni nada de eso. La idea es: un mundo tranquilo con hierba, árboles y un lago donde puedes caminar y encontrarte con aldeanos amigables que te saludan. También hay objetos regados por el mapa que puedes recoger.

No se cambiaron las clases base (Sprite, TileSet, Tile) porque funcionan igual. Lo que si se modificó fueron las clases que dependen de ellas y se agregaron dos completamente nuevas.

2. Descripción del videojuego

El juego se llama *Mi Aldea*. Es un mundo de 5000 x 5000 pixeles con cámara que sigue al jugador, igual que en la práctica anterior. La diferencia principal es el ambiente: en vez de una arena con columnas y murallas hay un pueblo con caminos de tierra, bosques de árboles, un lago y parches de flores.

El jugador puede moverse con las flechas del teclado, correr con Shift y recoger objetos del suelo presionando la barra espaciadora. Los objetos que se pueden recoger son manzanas, naranjas, flores, conchas y estrellas de mar. Cuando se recoge algo aparece una notificación con el nombre del objeto y se guarda en el inventario que se ve en la parte de abajo de la pantalla.

Los NPCs son los aldeanos: cinco simples que caminan por zonas del mapa y se acercan a saludar cuando el jugador pasa cerca, y un habitante especial llamado Toni-moqui que usa el algoritmo A* para llegar hasta el jugador esquivando obstáculos.

También hay un ciclo de día y noche. El color del cielo cambia lentamente entre noche, amanecer, día, atardecer y crepúsculo. Es un ciclo visual nada más, no afecta la jugabilidad.

3. Imágenes y recursos

3.1 Imágenes reutilizadas del proyecto anterior

Se reutilizaron dos archivos de imagen que ya teníamos:

- monitoB.png: el Sprite del personaje principal. No se cambió nada, funciona exactamente igual que antes.
- Arena_Tileset.png: se usa como base para dibujar el suelo del pueblo. Las filas 8 y 9 del tileset son los tiles de pasto/arena que se usan como hierba, y las filas 13 a 16 se usan para los caminos de tierra.

3.2 Gráficos generados por código

Los aldeanos, los árboles, el lago, las flores y los objetos recogibles se dibujan directamente con funciones de pygame (pygame.draw.circle, polygon, rect, ellipse). No necesitan archivos de imagen. Esto para no depender de recursos externos y que el juego funcione en cualquier sin buscar sprites adicionales.

3.3 Recursos opcionales para mejorar el juego

Si en el futuro se quisieran agregar sprites reales en vez de los generados por código, estas son las páginas donde se pueden encontrar de forma gratuita:

Sitio / Recurso	Descripción
kenney.nl	Tiles de RPG top-down, personajes, naturaleza. Todo es gratuito y sin restricciones de uso.
opengameart.org	Buscar "LPC tileset" para encontrar sprites estilo RPG 2D con licencia libre.

4. Estructura del código

El proyecto se divide en dos archivos igual que antes: Sprites_pueblo.py con todas las clases y mundo_pueblo.py con el bucle principal del juego.

4.1 Clases sin cambios

Estas tres clases se copiaron directamente del proyecto anterior porque no había razón para modificarlas:

- Sprite: la clase base que carga la imagen y define el rectángulo de colisión. Solo se le agrego un if para que no falle si el archivo de imagen no existe.
- TileSet: hereda de Sprite, carga el tileset y extrae los frames individuales.
- Tile: representa una celda del mapa con su posición, tipo e imagen. Igual que antes.

También se copió la función _astar() sin ningún cambio. El algoritmo es el mismo, con soporte para tiles de costo variable (tipo 1 = camino lento, costo x2).

4.2 TileSetPueblo (reemplaza a TileSetArena)

Esta clase hereda de TileSet igual que TileSetArena, pero genera un mapa de aldea en vez de una arena. La lógica interna es similar: se construye una cuadrícula, se asigna un tipo a cada celda y se dibujan los tiles encima.

Los tipos de tile son: 0 = hierba (libre), 1 = camino de tierra (costo x2 en A*), 2 = árbol (obstáculo), 3 = agua (obstáculo) y 4 = flores (decorativo, pisable).

El mapa se construye en cinco pasos: primero se llena todo de hierba, luego se traza una cruz de caminos, después se colocan grupos de árboles en distintas zonas, se genera un lago elíptico en el cuadrante noreste y por último se dispersan parches de flores aleatorios.

4.3 Aldeano (reemplaza a Murciélago)

El Aldeano es el equivalente al Murciélago del proyecto anterior pero amigable. Tiene la misma máquina de estados de tres estados: DEAMBULAR, SALUDAR y REGRESAR.

La diferencia principal es que no hereda de TileSet porque no usa un archivo de imagen. Se dibuja con pygame.draw. Cada aldeano tiene un color distinto y un nombre que se muestra encima de su cabeza. Cuando detecta al jugador cerca, cambia a estado SALUDAR, se acerca lentamente y muestra una burbuja de dialogo con un mensaje aleatorio.

Los aldeanos no usan A*. Se mueven directo hacia su destino. Si hay un obstáculo en el camino simplemente se detienen y esperan, igual que los murciélagos hacían con los límites de su zona.

4.4 Habitante (reemplaza a Gladiador)

El Habitante hereda de Aldeano igual que el Gladiador heredaba del Murciélago. La diferencia es que los estados SALUDAR y REGRESAR usan A* para moverse, entonces puede rodear obstáculos correctamente.

Recibe el mapa_celdas como parámetro para que A* sepa que tiles tienen costo adicional, exactamente igual que el Gladiador en la práctica anterior. Funciona de la misma manera, pero en vez de perseguir agresivamente al jugador, lo busca para saludarlo y después regresa a su zona.

4.5 Item (clase nueva)

Esta clase es completamente nueva, no tenía equivalente en el proyecto anterior. Un Item es un objeto recogible que aparece en el mapa: manzana, naranja, flor, concha o estrella de mar.

Cada uno tiene un tipo, una posición y una bandera recogido. Se dibuja con código (formas geométricas de colores). Tiene una animación de brillo que va y viene para que se note que es recogible. El jugador lo recoge presionando ESPACIO cuando está cerca.

4.6 Personaje (modificado)

El Personaje es básicamente el mismo que antes. Solo se le agregaron tres cosas: una lista inventario, la variable recoger_activo que se activa cuando se presiona ESPACIO, y el método recoger_item() que revisa si hay algún item cercano y lo mete al inventario.

5. Diagramas

En esta sección se incluyen los diagramas que describen la estructura y el comportamiento del programa. El diagrama de clases muestra como se relacionan las clases, las máquinas de estado describen el comportamiento de cada clase con estados, y el diagrama de flujo muestra como corre el juego.

5.1.1 Diagrama general

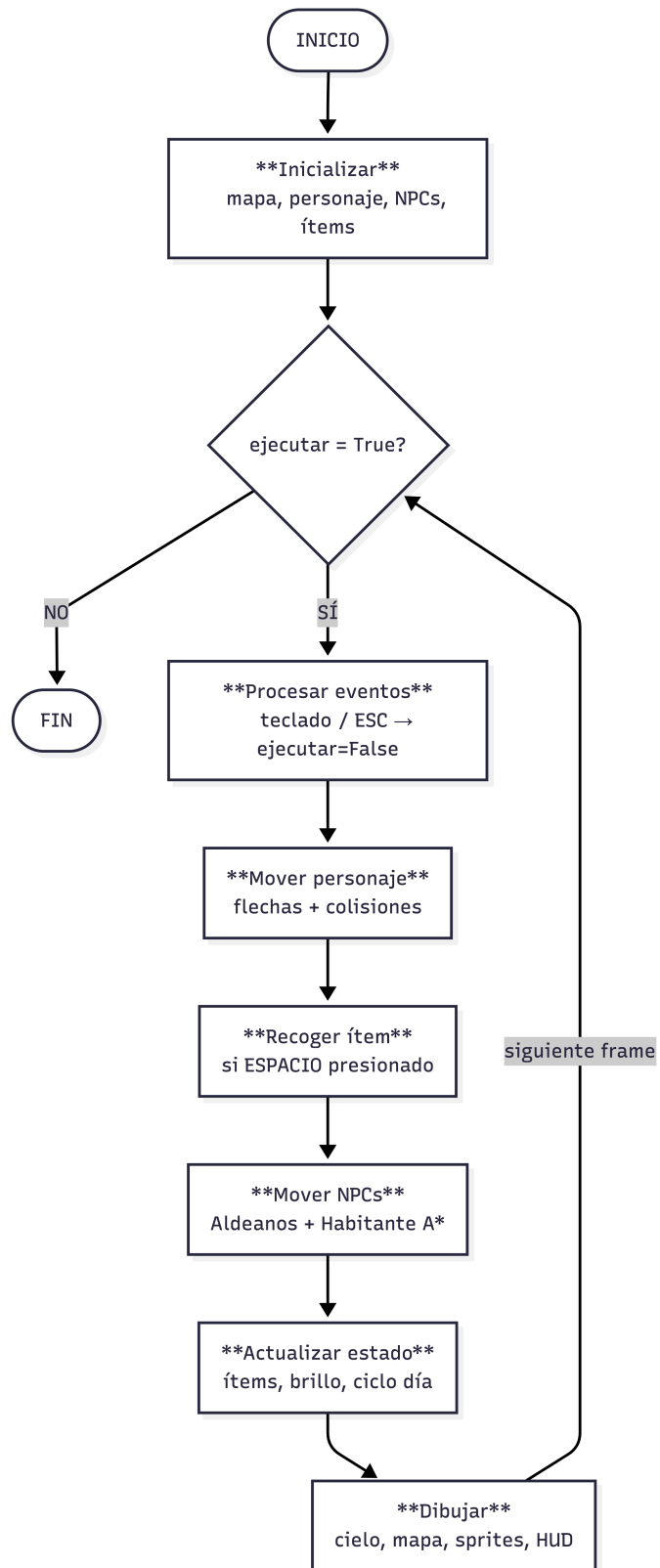


Figura 1. Diagrama de flujo general del bucle principal del juego

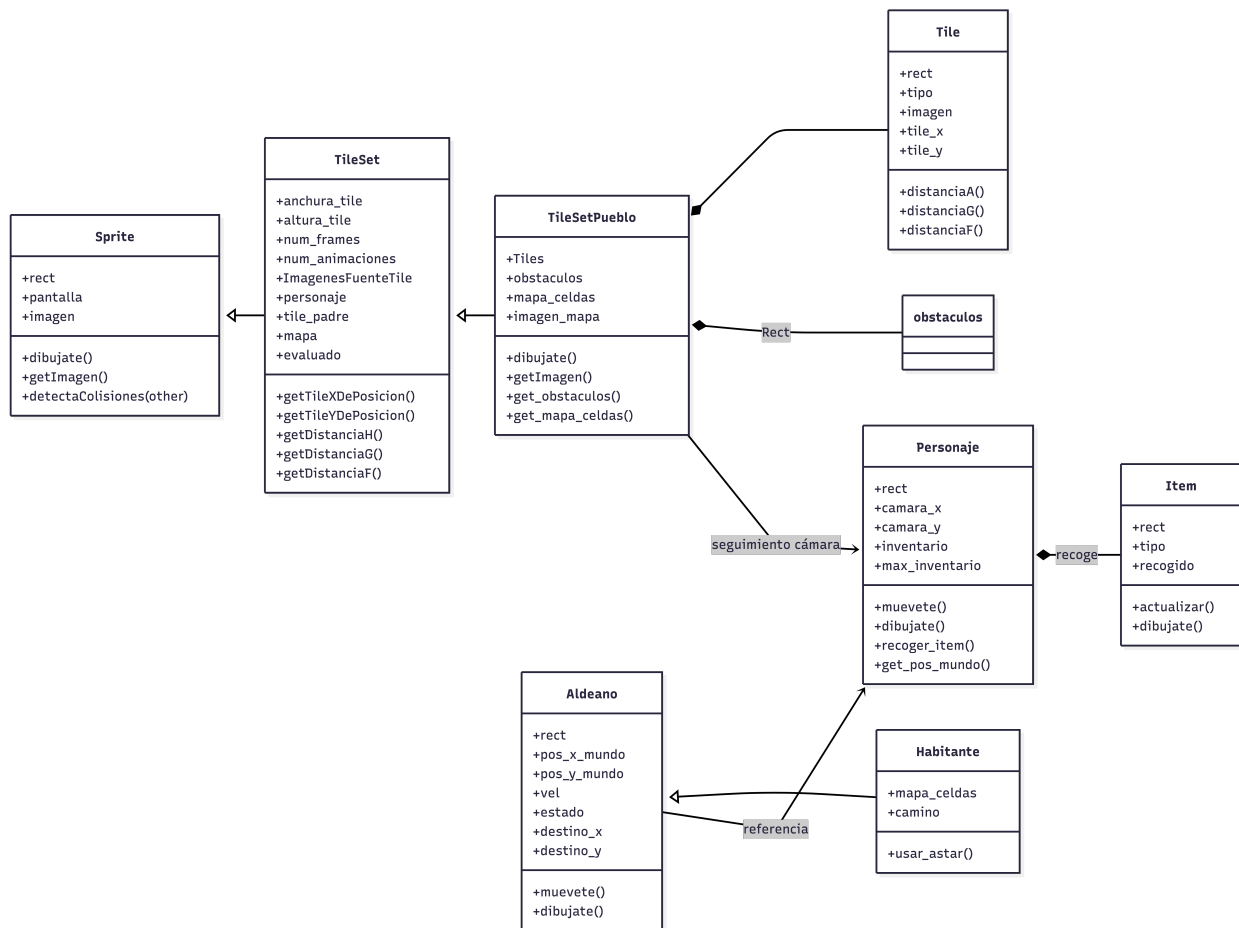


Figura 1.1 Diagrama de clases

5.1 Maquinas de estado

Las siguientes figuras muestran las maquinas de estado para las clases que tienen comportamiento por estados. Aldeano y Habitante comparten la misma estructura de tres estados, aunque Habitante usa A* internamente.

5.2.1 Aldeano

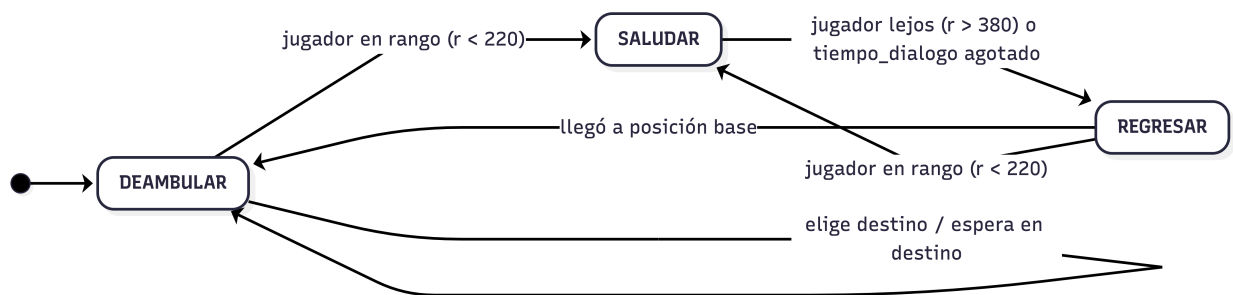


Figura 2.1. Maquina de estados — clase Aldeano

5.2.2 Habitante

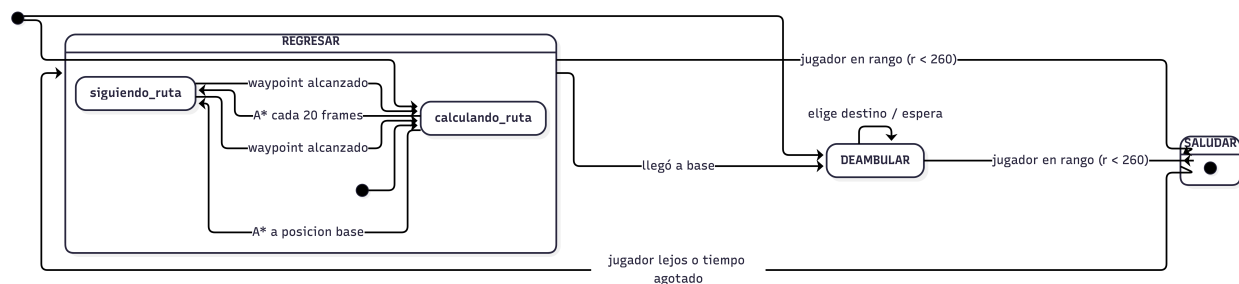


Figura 2.2 Máquina de estados — clase Habitante (con A* en SALUDAR y REGRESAR)

5.2.3 Personaje

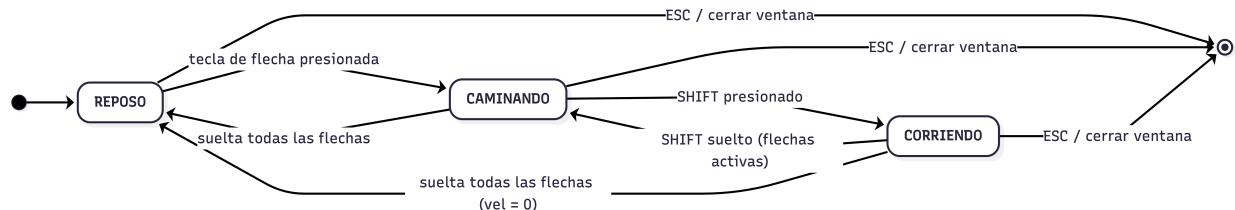


Figura 2.3. Máquina de estados — clase Personaje

5.2.4 Ítem

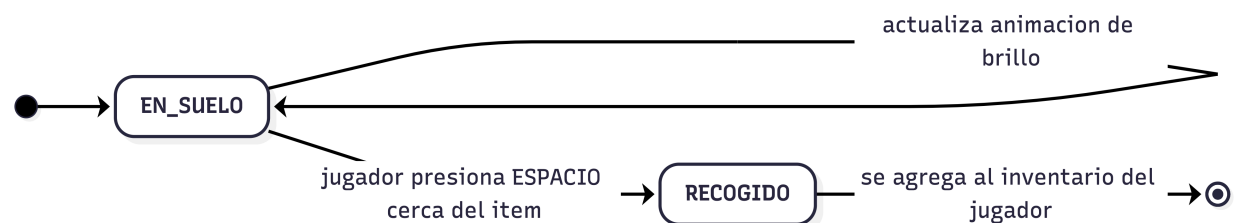


Figura 2.4. Máquina de estados — clase Ítem

6. Dificultades encontradas

La primera fue que los Aldeanos heredaran de SpriteAnimado para mantener la misma cadena de herencia del proyecto anterior. El problema es que SpriteAnimado hereda de TileSet y TileSet necesita cargar un archivo de imagen. Como los aldeanos se dibujan con código no hay imagen que cargar, y cada que intentaba crear un aldeano tronaba con un error de que no encontraba el archivo.

En Python no es obligatorio seguir exactamente la misma cadena de herencia. Si la clase nueva no necesita lo que da la clase padre, es mejor no heredar de ella y solo duplicar lo que si se necesita. Por eso Aldeano no hereda de nada, tiene sus propios atributos de posición y velocidad directamente.

La segunda fue el tiempo de carga del mapa. Con el mapa de 5000x5000 y todos los tiles dibujados individualmente, al principio tardaba como 20 segundos en cargar. El problema era que estaba llamando `getTileHierba()` dentro del ciclo de construcción y esa función usa `randint` cada vez, lo cual no es el problema real. El verdadero cuello de botella era hacer `Surface.blit()` miles de veces en el ciclo principal en vez de una sola vez al inicio. Cuando se cambió para pregenerar el mapa completo en un solo `Surface` al inicio, el tiempo de carga bajo a menos de 3 segundos. Eso es lo que hace `imagen_mapa`.

Otro detalle menor fue que los diálogos de los aldeanos tienen caracteres especiales como acentos. En Windows a veces esto causa problemas dependiendo del encoding del archivo. La solución fue simplemente quitarles los acentos a las frases del dialogo para evitar errores.

7. Conclusión

Esta práctica me ayudo a entender para que sirve la herencia en programación orientada a objetos. Es poder reutilizar comportamientos que ya funcionaban. El ciclo de día, los aldeanos y el sistema de inventario son cosas que no existían antes, pero el movimiento del personaje, el A* y la cámara funcionan igual que en el proyecto anterior.

La diferencia entre Aldeano y Habitante: los dos hacen lo mismo visualmente, pero uno llega a su destino en línea recta y el otro rodea los obstáculos.

Se puede agregar música de fondo y que los aldeanos digan cosas diferentes según la hora del día. También se podría actualizar a los ítems para reaparecer con el tiempo y que el mapa no se quede vacío.